Documentación del uso de la IA - Colorfly Studio

Documentación del proceso de desarrollo

Nombre: Ana Márquez Almazán

Materia / Institución: 1er entrega M1 fullstack

Fecha: 8 de abril del 2026

---

1. Introducción

Colorfly Studio es una aplicación web que genera paletas de colores aleatorios. El usuario elige cuántos colores quiere (6, 8 o 9), presiona el botón Generar y la app crea tarjetas mostrando cada color en formato HEX, RGB y HSL.

Objetivo del proyecto:

El aprendizaje de conceptos básicos de HTML5, CSS, Javascript, Git y GitHub. Empleando estos conocimientos para crear esta aplicación que genera colores de manera aleatoria.

---

2. Análisis del problema

Antes de escribir código, se realizó un análisis del problema definiendo los pasos necesarios para construir la aplicación.

Pseudocódigo inicial:

```
// Usuario elige un número
// Guardar el número en una variable
// Repetir N veces:
//   → Generar un color HEX aleatorio
//   → Guardarlo en una lista
```

// Mostrar cada color como una caja en pantalla

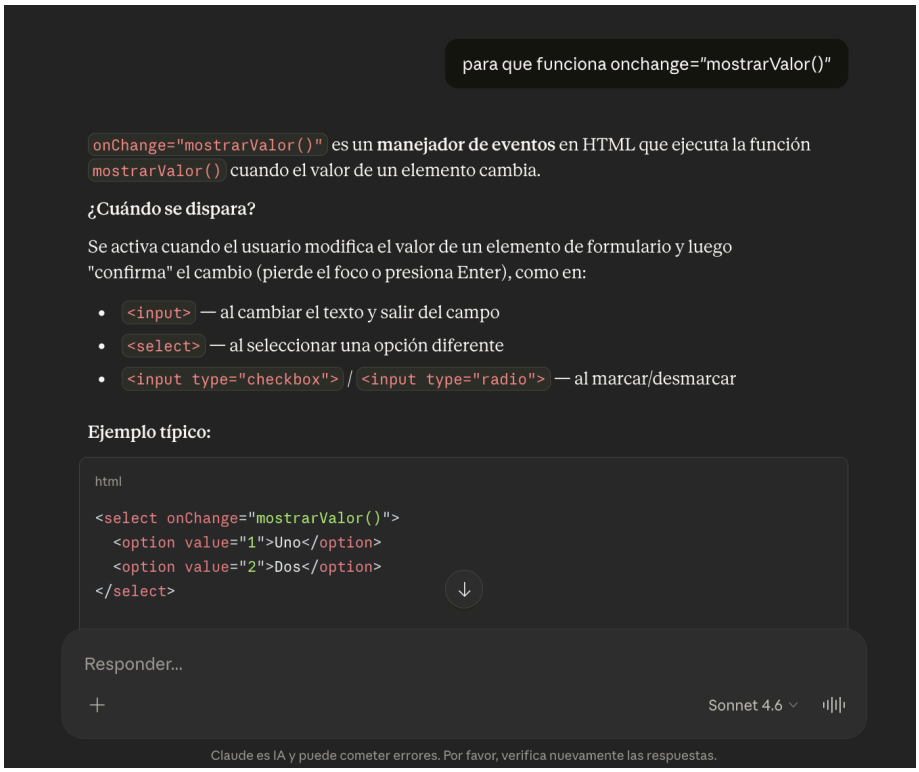
Tecnologías utilizadas: HTML5, JavaScript (Vanilla), CSS

---

3. Bitácora de desarrollo

En esta sección se documenta cada consulta de AI realizada durante el desarrollo, indicando la fuente, lo aprendido y cómo se aplicó al código.

3.1 Evento onChange y manejo de eventos

Consulta	*para que funciona onchange="mostrarValor()"*
Fuente	*Claude*
Captura de pantalla	

Lo que aprendí

Aprendí que `onChange` es un evento que se dispara automáticamente cuando el usuario cambia el valor de un elemento como un `<select>`. La diferencia con leer el valor directamente es que `onChange` lo detecta en el momento exacto del cambio, mientras que si guardas el valor al cargar la página, la variable no se actualiza sola. También aprendí que para un botón el evento equivalente es `onclick`.

Cómo lo apliqué

Al inicio lo usé en el `<select>` para detectar cuántos colores quería generar el usuario. Después entendí que como tengo un botón Generar, no necesitaba `onChange` en el select — el botón lee el valor en el momento que el usuario hace clic, así que lo eliminé del HTML.

3.2 Crear elementos HTML con createElement

Consulta

Si quiero que cree el div al dar clic a un boton...en el siguiente codigo no me des la respuesta hazme pensar que estoy aprendiendo

Fuente

Claude

Captura de pantalla

Veo tu código, ¡vas muy bien! Te hago preguntas para que lo descubras tú:

Error 1 — esta línea está de más:

```javascript
let newColor = document.createElement("gid"); // ← antes del for
```

¿Por qué crees que sobra? Pista: mira qué pasa con `newColor` dentro del `for` 👀

Error 2 — esta línea está en el lugar equivocado:

```javascript
grid.innerHTML = ""; // limpia las cards anteriores
```

¿Debería estar dentro o fuera del `for`? Piensa: si está dentro del `for`, ¿qué pasaría en cada repetición? 🤔

Pregunta sobre el botón:

Ahora mismo tienes la función `saveValue()`, pero... ¿dónde está el botón en tu HTML que la llama? ¿Qué atributo necesita ese botón para ejecutar la función al hacer clic?

Pista: en el ejemplo anterior usamos `onchange="..."` en el select. ¿Cuál sería el equivalente para un botón? 🙂

Respóndeme esas preguntas y vamos avanzando paso a paso.

¡Muy bien, vas mejorando! Te hago pensar con preguntas:

Sobre el `onchange` en el select:

Ahora tienes AMBOS al mismo tiempo:

- `onchange="saveValue()"` en el select
- `onclick="saveValue()"` en el botón

¿Qué problema crees que causa tener los dos? Piensa: ¿cuántas veces se ejecutaría `saveValue()` si el usuario cambia el select Y luego presiona el botón? 🤔

Hay un error que aún persiste aquí:

```javascript
newColor = document.createElement("grid"); // ← qué va aquí adentro?
```

`"grid"` no es una etiqueta HTML válida. ¿Qué etiqueta HTML usamos para crear una caja o contenedor? 👀

Lo que aprendí

Aprendí que `document.createElement()` sirve para crear elementos HTML desde JavaScript sin tener que escribirlos directamente en el HTML. Aprendí también que `classList.add()` le agrega clases CSS al elemento recién creado, y que `innerHTML` le pone contenido adentro.

Cómo lo apliqué Lo usé dentro del `for` de la función `saveValue()` para crear un `div` por cada color generado.

3.3 El ciclo for y repetición de acciones

Consulta *pero veo un problema los está generando de nuevo y aumentando a pesar de que elimine onchange*

¿para qué sirve un for javascript?

Fuente *Claude / Google*

Captura de pantalla

¡Buen ojo que lo detectaste! Te pregunto:

Cada vez que presionas el botón, el `for` crea divs nuevos y los agrega al grid... pero **¿en algún lugar de tu código estás borrando los divs anteriores antes de crear los nuevos?**

Teníamos esta línea en el ejemplo anterior:

```javascript
grid.innerHTML = "";
```

¿Dónde crees que debería ir esa línea dentro de tu función? 👀

Y un segundo problema chiquito, mira tu `for`:

```javascript
for (let i = 0; i <= miSelectColor; i++)
```

Si `miSelectColor` es 6 y empiezas desde `0`... ¿cuántas veces se repite realmente? Cuenta: 0,1,2,3,4,5,6 — ¿cuántas son? 🤔

¿Debería empezar en `0` o en `1`?

¡Dos pistas para que lo descubras!

Problema 1 — el orden dentro del `for`:

Mira qué pasa en cada vuelta del `for`:

1. Crea el div ✅
2. Le agrega clases ✅
3. Lo agrega al grid ✅
4. **Borra todo el grid** 💥

¿Ves el problema? El `grid.innerHTML = ""` está **dentro** del `for` y se ejecuta en cada vuelta. ¿Dónde debería estar, dentro o fuera del `for`? 🤔

Problema 2 — ya te lo había mencionado antes:

```javascript
newColor = document.createElement("grid"); // ← "grid" no es HTML
```

`"grid"` no existe como etiqueta HTML. ¿Qué etiqueta sí existe para crear una caja? La usaste correctamente en versiones anteriores de tu código 👀

Corrígelos y cuéntame qué pasa 🙂

Visión general creada por IA

Un bucle `for` en JavaScript sirve para **repetir un bloque de código un número específico de veces o hasta que se cumpla una condición**. Es fundamental para automatizar tareas repetitivas, iterar sobre arrays (listas) o recorrer propiedades de objetos. MDN Web Docs +4

Lo que aprendí

Aprendí que el `for` tiene siempre la misma estructura: un valor inicial, una condición de hasta dónde llegar, y un incremento. Lo importante es entender bien esos tres valores porque determinan cuántas veces se repite el código. También descubrí que si empiezas en `0` en vez de `1` el ciclo se repite una vez de más, y que `grid.innerHTML = ""` debe estar **fuera** del `for` para limpiar las cards anteriores solo una vez, no en cada vuelta.

Cómo lo apliqué

Lo usé para crear los `div` necesarios según el número que el usuario seleccionó en el `<select>`. En cada vuelta del `for` se crea una card nueva con su color, su HEX, RGB y HSL.

3.4 parseInt y conversión de bases numéricas

Consulta

encontre lo siguiente pero tu me dices que debo convertir hexadecimal a rgb y despues a hsl

In JavaScript, you use the `parseInt()` function with a second argument of `16` to convert a hexadecimal string to a decimal integer. The second argument, known as the radix, specifies the base of the number being parsed. Usage

Fuente

Claude

Captura de pantalla

¡Exacto, lo encontraste sola! 🎉

Entonces ya sabes convertir HEX a número normal:

```javascript
parseInt("A3", 16) // → 163
parseInt("F2", 16) // → 242
parseInt("C1", 16) // → 193
```

Siguiente pregunta 🤔

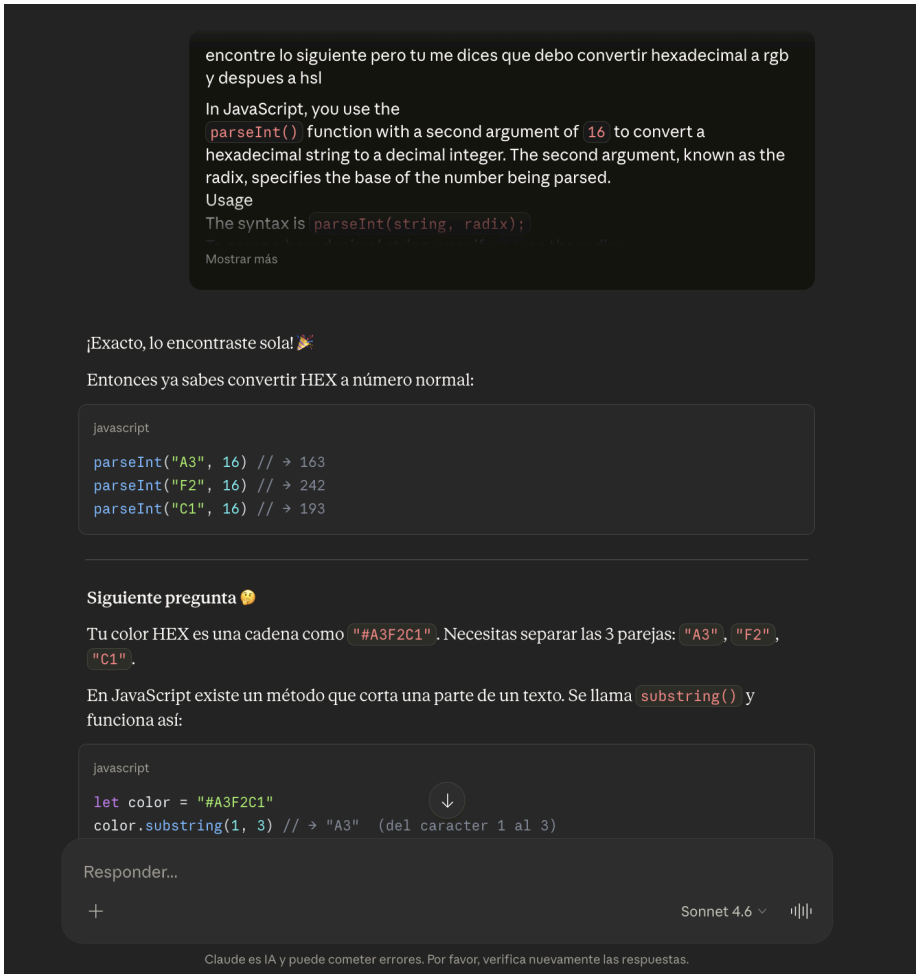
Tu color HEX es una cadena como `"#A3F2C1"`. Necesitas separar las 3 parejas: `"A3"`, `"F2"`, `"C1"`.

En JavaScript existe un método que corta una parte de un texto. Se llama `substring()` y funciona así:

```javascript
let color = "#A3F2C1"
color.substring(1, 3) // → "A3"  (del caracter 1 al 3)
color.substring(3, 5) // → "F2"  (del caracter 3 al 5)
color.substring(5, 7) // → ???   (¿qué crees que da?)
```

¿Puedes adivinar qué regresa `color.substri`[illegible]`, 7)`? 😉

Y otra pregunta: ¿por qué empieza en `1` y no en `0`? Mira el color `"#A3F2C1"` y cuenta los



Lo que aprendí

Aprendí que `parseInt()` recibe dos argumentos: el texto a convertir y el **radix**, que es la base del sistema numérico en el que está escrito ese texto. Al pasarle `16` como segundo argumento le estamos diciendo a JavaScript que el texto está en hexadecimal y que lo convierta a un número decimal normal. Por ejemplo `parseInt("A3", 16)` devuelve `163`. Esto es fundamental para la conversión de HEX a RGB porque un color como `#A3F2C1` necesita separarse en tres parejas (`A3`, `F2`, `C1`) y cada una convertirse a su valor numérico equivalente entre 0 y 255.

Cómo lo apliqué

Lo usé dentro de la función `hexToRGB()` junto con `substring()` para extraer cada pareja de caracteres del color HEX y convertirla a su valor RGB correspondiente:

3.5 substring y manipulación de texto

Consulta ¿Por qué lleva el número 16? En `toString(16)`

Fuente *Claude*

Captura de pantalla

¡Exacto, lo encontraste sola! 🎉

Entonces ya sabes convertir HEX a número normal:

```javascript
parseInt("A3", 16) // → 163
parseInt("F2", 16) // → 242
parseInt("C1", 16) // → 193
```

Siguiente pregunta 🤔

Tu color HEX es una cadena como `"#A3F2C1"`. Necesitas separar las 3 parejas: `"A3"`, `"F2"`, `"C1"`.

En JavaScript existe un método que corta una parte de un texto. Se llama `substring()` y funciona así:

```javascript
let color = "#A3F2C1"
color.substring(1, 3) // → "A3"  (del caracter 1 al 3)
color.substring(3, 5) // → "F2"  (del caracter 3 al 5)
color.substring(5, 7) // → ???   (¿qué crees que da?)
```

¿Puedes adivinar qué regresa `color.substri` ↓ `, 7)`? 😉

Y otra pregunta: ¿por qué empieza en `1` y no en `0`? Mira el color `"#A3F2C1"` y cuenta los

Lo que aprendí Aprendí que el `16` en `toString(16)` indica el sistema numérico al que quieres convertir el número. El sistema hexadecimal usa la base 16 porque tiene 16 caracteres posibles: los números del 0 al 9 y las letras A, B, C, D, E, F. Los colores HEX como `#A3F2C1` usan exactamente ese sistema. También aprendí que `toString()` acepta diferentes bases — por ejemplo `toString(2)` convierte a binario y `toString(10)` a decimal normal. El número `16777215` equivale a `#FFFFFF` en hexadecimal, que es el blanco puro, y representa todos los colores posibles.

Cómo lo apliqué Lo usé en la función `generateNewColor()` para construir colores HEX aleatorios carácter por carácter.

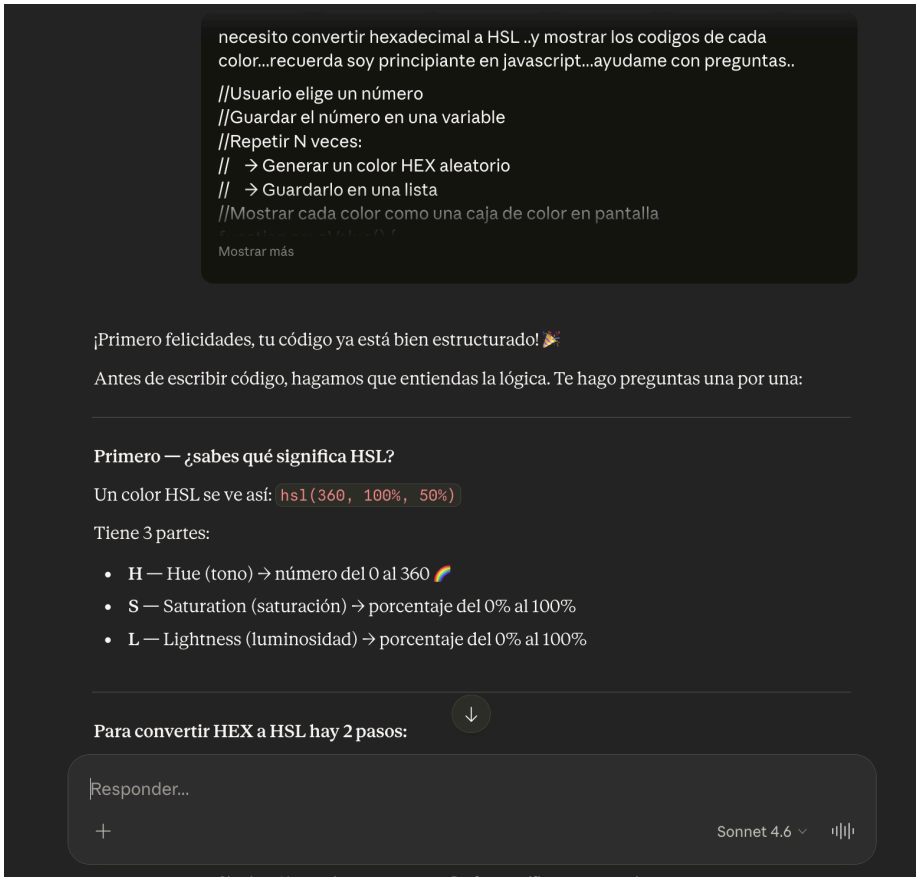
3.6 Conversión de RGB a HSL

Consulta *necesito convertir hexadecimal a HSL …y mostrar los codigos de cada color...recuerda soy principiante en javascript..ayudame con preguntas*

como puedo convertir rgb o HEX a HSL...cual seria mas facil??

Fuente *Claude*

Captura de pantalla



- **L** — Lightness (luminosidad) → porcentaje del 0% al 100%

Para convertir HEX a HSL hay 2 pasos:

```
HEX → RGB → HSL
```

Primero conviertes `#A3F2C1` a RGB, luego el RGB a HSL.

Pregunta para ti 🤔

Un color HEX como `#A3F2C1` tiene 6 caracteres después del `#`. Se dividen en 3 parejas:

```
#  A3  F2  C1
   R   G   B
```

¿Puedes adivinar qué representa cada pareja? 😊

como puedo convertir rgb o HEX a HSL...cual seria mas facil??

¡Buena pregunta! RGB a HSL es más fácil porque ya tienes los valores `r`, `g`, `b` listos en `miRGB` 😊

La conversión tiene una lógica matemática, te la explico paso a paso:

HSL necesita 3 cosas:

- **H** (tono) → 0 a 360
- **S** (saturación) → 0% a 100%
- **L** (luminosidad) → 0% a 100%

Paso 1 — Los valores RGB van de 0 a 255, pero para la fórmula necesitan ir de 0 a 1. ¿Cómo convertirías `255` a `1`? 🤔

Pista:

javascript

```javascript
let r = miRGB.r / ???
let g = miRGB.g / ???
let b = miRGB.b / ???
```

Responder...

Sonnet 4.6

Lo que aprendí

Aprendí que la conversión más fácil es RGB → HSL porque ya tenía los valores separados. Entendí que los colores HEX usan el sistema de base 16, y que para convertirlos primero hay que separar los 6 caracteres en 3 parejas (R, G, B) usando `substring()` y convertirlas a números normales con `parseInt("A3", 16)`.

Para calcular HSL aprendí que primero hay que convertir los valores RGB de escala 0–255 a escala 0–1 dividiendo entre 255. Luego se busca el valor máximo y mínimo con `Math.max()` y `Math.min()`

para calcular la luminosidad (L), la saturación (S) y el tono (H). El tono se multiplica por 60 para convertirlo a grados (0–360°). Descubrí que H puede salir negativo, y para corregirlo se le suma 360.

Cómo lo apliqué

Creé dos funciones: `hexToRGB()` que convierte el color HEX generado a sus valores R, G y B, y `rgbToHSL()` que toma esos tres valores y calcula H, S y L. Ambas funciones las uso dentro del `for` de `saveValue()` para mostrar los tres códigos en cada card de color.

3.7 Scope (alcance) de funciones en JavaScript

Consulta *Si entiendo pero aquí está fuera del alcance la variable*

Fuente *Claude*

Captura de pantalla

Lo que aprendí

Aprendí que en JavaScript cada función tiene su propio alcance (scope), lo que significa que las variables y funciones declaradas dentro de una función no son accesibles desde afuera. Si una función necesita usar algo que está dentro de otra, no puede acceder a ello. La solución es declarar las funciones y variables al mismo nivel, fuera de cualquier otra función, como "vecinas" y no como "muñecas rusas" una dentro de la otra.

Cómo lo apliqué

Moví `generateNewColor()` fuera de `saveValue()` para que pudieran ser usadas desde cualquier parte del código. Así `saveValue()` pudo llamar a `generateNewColor()` sin problemas.

---

4. Resultado final

Entregable Final

Repositorio en GitHub

Repositorio: https://github.com/anakayni/ProyectoM1_Ana_MarquezAlmazan.git

---

5. Conclusiones

¿Qué aprendí en este proyecto?

Este proyecto me permitió aprender JavaScript desde cero de forma práctica. Aprendí a manipular el DOM creando y modificando elementos HTML dinámicamente con `createElement`, `classList` y `innerHTML`. Entendí cómo funcionan las funciones, cómo se llaman entre sí y la importancia del scope para que puedan accederse correctamente. También aprendí el sistema de numeración hexadecimal y cómo los colores se representan en tres formatos diferentes: HEX, RGB y HSL, incluyendo las fórmulas matemáticas para convertir entre ellos. Además practiqué el uso de ciclos `for`, eventos como `onclick`, y el manejo de estilos CSS directamente desde JavaScript.

¿Qué fue lo más difícil?

La parte más difícil fue comprender la lógica detrás de los colores: cómo se generan, cómo se convierten entre formatos y por qué la conversión a HSL puede producir valores negativos. Para superarlo le pedí a la IA ejemplos muy prácticos y concretos que me ayudaron a entender cada paso por separado antes de juntarlos.

¿Qué mejorarías en el futuro?

Me gustaría seguir desarrollando Colorfly Studio agregando nuevas funcionalidades como guardar paletas de colores y eliminarlas con un botón, copiar el código de un color al portapapeles con un solo clic, fijar un color para que no cambie al generar una nueva paleta, y seleccionar el tipo de paleta a generar como análoga o complementaria como lo vi en otros sitios.